

fxml之二



JavaFX控件属性的设置方法

北京理工大学计算机学院
金旭亮

JavaFX控件属性的设置方式



设置控件的属性（property），可以使用XML元素的属性（attribute），也可以使用XML子元素。

1

```
<Label>  
    <text>Hello, World!</text>  
</Label>
```

子元素设置方式

2

```
<Label text="Hello, World!"/>
```

元素属性设置方式



在FXML文档中，凡是小写字母开头的元素，通常都对应着JavaFX控件的属性。

复杂的属性类型



有一些属性，本身是一个复杂类型，这时，可以使用属性元素嵌套的方式进行设置。

```
<VBox alignment="CENTER" spacing="20.0" xmlns:fx="http://javafx.com/fxml"
      fx:controller="com.jinxuliang.fxmlsetproperty.MainController">
  <padding>
    <Insets bottom="20.0" left="20.0" right="20.0" top="20.0"/>
  </padding>
```



上述代码中，VBox控件的padding属性，是Insets对象，因此，可以元素嵌套的方式进行定义。

默认属性可以被省略



```
<VBox spacing="20">  
  <children>  
    <Label text="Line 1"/>  
    <Label text="Line 2"/>  
  </children>  
</VBox>
```

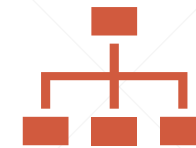


```
<VBox spacing="20">  
  <Label text="Line 1"/>  
  <Label text="Line 2"/>  
</VBox>
```

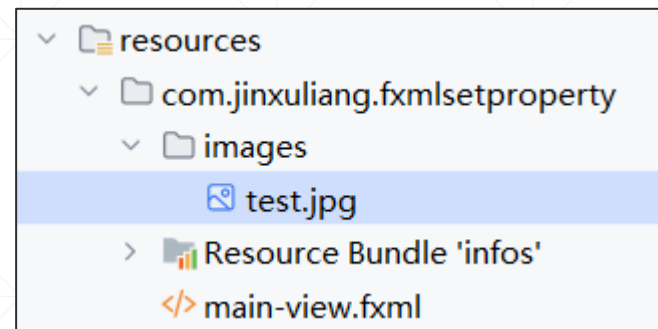


在JavaFX控件中，那些标识有`@DefaultProperty(value="属性名")`注解的属性，是默认属性。

"@"前缀



```
<!--使用@指定相对路径-->  
<ImageView>  
    <Image url="@images/test.jpg"/>  
</ImageView>
```

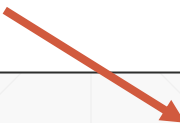


"@"表示相对路径。比如，上面的fxml声明中，表示test.jpg这张图片，则位于本fxml所在文件夹的images子文件夹下。

使用常量定义属性值

```
<VBox alignment="CENTER" spacing="20.0"
      xmlns:fx="http://javafx.com/fxml"
      fx:controller="com.jinxuliang.fxmlsetproperty.MainController">
  <padding...>

  <!--在属性中使用常量-->
  <alignment>
    <!-- 使用JavaFX内置的常量-->
    <Pos fx:constant="CENTER"/>
  </alignment>
  <Label style="-fx-font-weight: bold;-fx-font-size: 20">
    <!--使用自定义类型中的常量-->
    <MyClass fx:constant="MAX_VALUE"/>
  </Label>
```



```
public class MyClass {
    1 usage
    public static final int MAX_VALUE = 255;
}
```

静态属性

静态属性，可以在一个控件内部，定义另一个控件的相关属性。

```
<!-- 使用静态属性-->
<GridPane>
  <Label text=" (0,0) ">
    <GridPane.rowIndex>0</GridPane.rowIndex>
    <GridPane.columnIndex>0</GridPane.columnIndex>
  </Label>
  <Label text=" (0,1) ">
    <!-- 使用静态属性-->
    <GridPane.rowIndex>0</GridPane.rowIndex>
    <GridPane.columnIndex>1</GridPane.columnIndex>
  </Label>
</GridPane>
```

表达式绑定 (Expression Binding)



在FXML中，使用“\$”定义数据绑定表达式：

```
<TextField fx:id="textField"/>  
<Label text="${textField.text}"/>
```



这是一个与Label绑定的文本框控件

这是一个与Label绑定的文本框控件

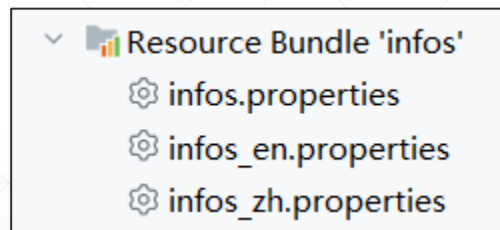
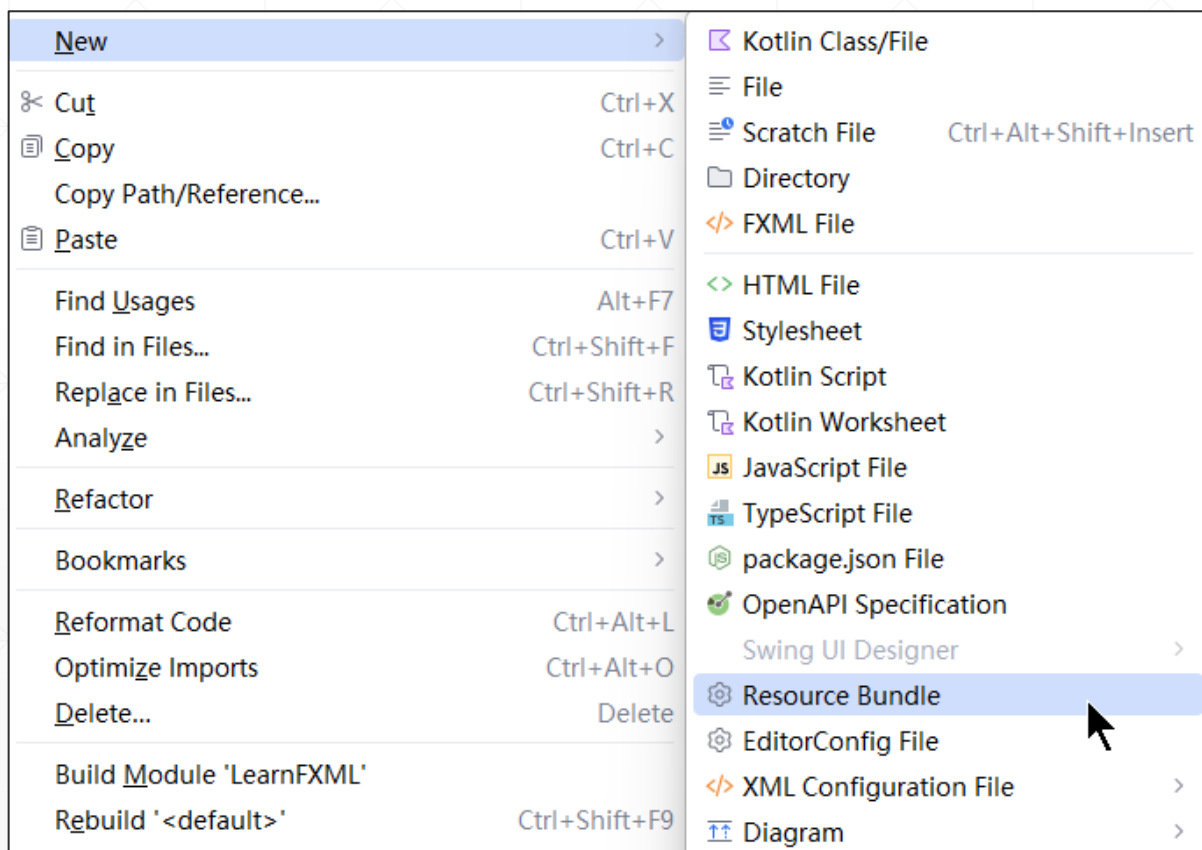


上面的代码，让文本框控件与标签控件之间实现了数据同步，也就是说，当用户在文本框中输入时，其输入的文本，会即时地同步显示于标签控件中。

使用资源包

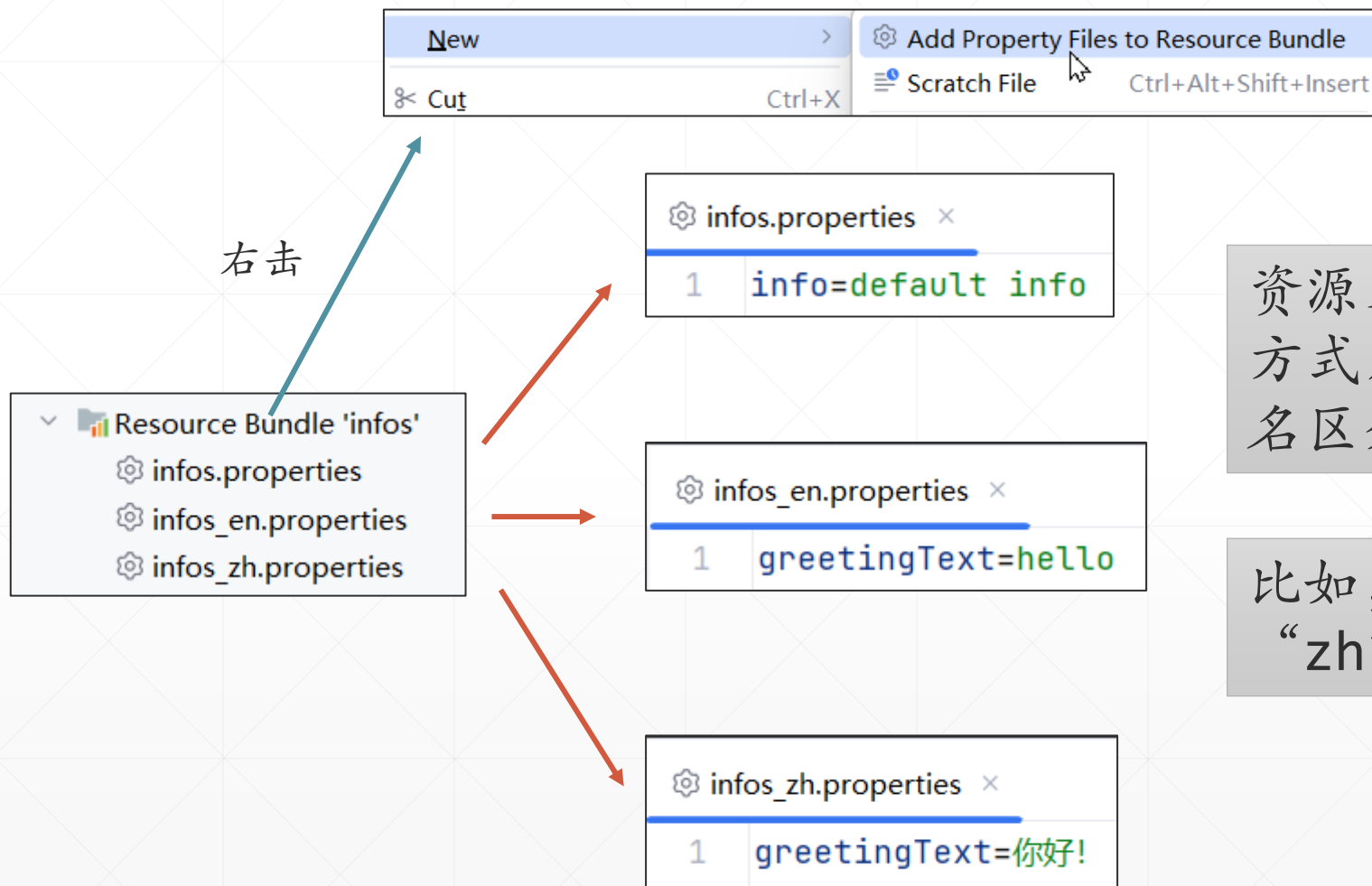


如果JavaFX应用需要支持多种语言，则可以向项目资源文件夹中添加“**资源包 (Resource Bundle)**”。



一个资源包中，可以放置多个 **properties** 资源文件，每种资源文件对应一种语言。

定义资源文件



资源文件以“key=value”方式定义字符串，通过文件名区分语言。

比如，“en”表示英语，“zh”表示汉语.....

引用资源字符串

```
<!--引用放在默认资源包中的字符串-->  
<Label text="%info"/>  
<!--可以动态地设置要使用的语言-->  
<Label text="%greetingText"/>
```

default info

你好!

中文

default info

hello

英文

```
@Override  
public void start(Stage stage) throws IOException {  
  
    //指定加载中文资源  
    Locale locale = new Locale( language: "zh");  
    //这里也可以指定使用英文资源  
    //locale = new Locale("en");  
    Locale.setDefault(locale);  
    //注意一下资源包的位置  
    var resourceBundle = ResourceBundle.getBundle(  
        baseName: "com.jinxuliang.fxmlsetProperty.infos");  
    FXMLLoader fxmlLoader = new FXMLLoader(  
        FxmlPropertyApp.class.getResource( name: "main-view.fxml"),  
        resourceBundle  
    );  
  
    Scene scene = new Scene(fxmlLoader.load());  
    stage.setTitle("设置fxml元素属性");  
    stage.setScene(scene);  
    stage.show();  
}
```

有几种语言?

```
public class LocaleInfo {  
    no usages  
    public static void main(String[] args) {  
        Locale[] localeList = Locale.getAvailableLocales();  
        //遍历数组的每个元素, 依次获取所支持的国家 and 语言  
        for (int i = 0; i < localeList.length; i++) {  
            var locale=localeList[i];  
            var country=locale.getCountry();  
            var displayCountry= locale.getDisplayCountry();  
            var language=locale.getLanguage();  
            var displayLanguage=locale.getDisplayLanguage();  
            if(country.length()>0)  
                System.out.printf("country: %s | displayCountry: %s %n",  
                                    country,displayCountry);  
            if(language.length()>0)  
                System.out.printf("language: %s | displayLanguage: %s %n",  
                                    language,displayLanguage);  
            System.out.println();  
        }  
    }  
}
```

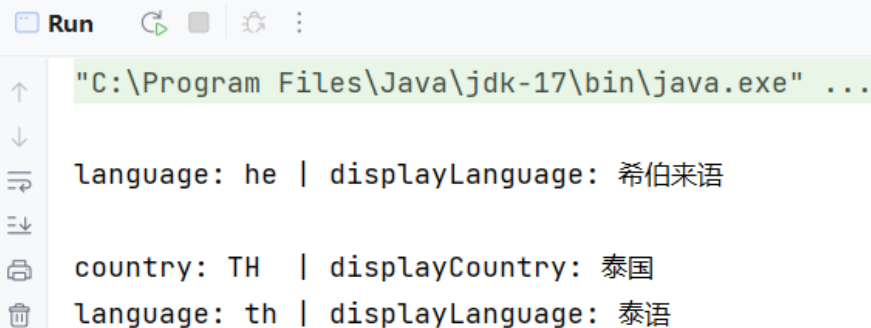
可以使用左边的代码, 了解当前系统支持的语言与国家。

按照以下格式定义针对特定国家的资源:

文件名_语言名_国名.properties

比如, 以下文件适用于简体中文:

文件名_zh_CN.properties



```
Run  "C:\Program Files\Java\jdk-17\bin\java.exe" ...  
language: he | displayLanguage: 希伯来语  
country: TH | displayCountry: 泰国  
language: th | displayLanguage: 泰国
```